

Reviewer Pack — Minimal Order of p-adic Integers and DPLL Proof Length Inequ...

Entry f4047dad268 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 04:59:11 UTC

Minimal Order of p-adic Integers and DPLL Proof Length Inequality

Entry ID: f4047dad268 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 04:59:11 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis **Field B** (complexity object): DPLL Proof Complexity

Statement:

For every CNF φ with n variables, the minimal order of a primitive element in the finite field extension of $\mathbb{Q}_p$ corresponding to the set of prime numbers dividing the DPLL proof length of φ is upper-bounded by $O(n^{1/p})$ where p is the smallest prime not dividing the number of clauses in φ .

Rationale (proposer's reasoning):

p-adic analysis provides a framework for studying arithmetic properties that could be related to the complexity of computational problems like SAT. This conjecture suggests that certain arithmetic properties of the underlying field may provide insights into the proof length required by DPLL solvers, potentially revealing deeper connections between number theory and computational complexity.

Taxonomy category: p_adic_analysis (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7e4c2ad9e4dd3cad

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The minimal order of a primitive element in $\mathbb{Q}_p$ is upper-bounded by $O(n^{1/p})$ for CNF φ with n variables and smallest prime p not dividing clause count.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        cnf = []
        for _ in range(2 * n):
            clause = [random.randint(-n, -1), random.randint(1, n)]
            if len(set(clause)) == 2:
                cnf.append(clause)
        return cnf

    def dpll_proof_length(cnf):
        # Simplified DPLL algorithm to estimate proof length
        stack = []
        literals = set()
        for clause in cnf:
            literals.update(abs(l) for l in clause)

        def simplify():
            while True:
                unit_clauses = [l for l in literals if abs(l) not in literals]
                if not unit_clauses:
                    break
                literal = unit_clauses[0]
                stack.append((literal, 'unit'))
                literals.remove(abs(literal))
                cnf = [c for c in cnf if literal not in c and -literal not in c]

        def propagate():
            while True:
                pure_literals = [l for l in literals if sum(1 for c in cnf if l in c) == 0 or sum(
                    if not pure_literals:
                        break
```

```

        literal = pure_literals[0]
        stack.append((literal, 'pure'))
        literals.remove(abs(literal))
        cnf = [c for c in cnf if literal not in c and -literal not in c]

    simplify()
    propagate()
    return len(stack)

def smallest_prime_not_dividing(k):
    p = 2
    while True:
        if k % p != 0:
            return p
        p += 1

n = random.randint(5, 40)
cnf = generate_cnf(n)
proof_length = dpll_proof_length(cnf)
clause_count = sum(len(clause) for clause in cnf)
p = smallest_prime_not_dividing(clause_count)

if p == 1:
    return {
        "metric_name": "minimal_order",
        "metric_value": None,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

def finite_field_extension(p):
    # Simplified representation of a finite field extension
    return p

def primitive_element_order(p):
    # Order of a primitive element in the finite field  $\mathbb{Q}_p$ 
    if p == 2:
        return 1
    for i in range(1, p):
        if pow(i, p - 1, p) == 1:
            return i

order = primitive_element_order(finite_field_extension(p))

if order is None:
    return {
        "metric_name": "minimal_order",
        "metric_value": None,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }
}

```

```

upper_bound = n ** (1 / p)
ratio = order / upper_bound

return {
    "metric_name": "minimal_order",
    "metric_value": ratio,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": ratio <= 1,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean_value = sum(r['metric_value'] for r in results if r['metric_value'] is not None) / len(results)
    std_value = math.sqrt(sum((r['metric_value'] - mean_value) ** 2 for r in results if r['metric_value'] is not None) / len(results))
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r['conjecture_holds'])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1c5e5c7e.py", line 125, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1c5e5c7e.py", line 69, in run_trial
    proof_length = dpll_proof_length(cnf)
                  ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1c5e5c7e.py", line 57, in dpll_proof_length
    propagate()
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1c5e5c7e.py", line 48, in propagate
    pure_literals = [l for l in literals if sum(1 for c in cnf if l in c) == 0 or sum(1 for c in cnf if -l in c) == 0]
                                     ^^^
UnboundLocalError: cannot access local variable 'cnf' where it is not associated with a value

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture's claim. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16683
2	preregistration	ollama_remote	glm4:latest	0	0	14058
3	novelty	ollama_remote	glm4:latest	0	0	8596
4	novelty	ollama_remote	glm4:latest	0	0	8854
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	35762
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12429
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11812
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14737
9	judge	ollama_remote	glm4:latest	0	0	11494

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 134424 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/f4047dadc268.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f4047dadc268.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f4047dadc268.tar.gz` (if generated)